

Project Card

Project: **Cat's Odyssey**

Purpose:

Cat's Odyssey es un videojuego 2D de plataformas y puzzles para PC, protagonizado por Preciosa, una gata que atraviesa escenarios misteriosos con el objetivo de regresar con su dueña. El jugador podrá explorar libremente, saltar, trepar o adherirse a ciertas superficies, mover objetos para resolver puzzles, evitar enemigos, recoger coleccionables y activar checkpoints. La narrativa tendrá un tono triste, misterioso y emocional.

Stakeholder Card

Project: **Cat's Odyssey**

Stakeholder: **Usuario / Jugador**

Goal:

Completar los niveles de Cat's Odyssey mediante controles comprensibles, plataformas y puzzles, siguiendo la historia de Preciosa sin que problemas técnicos dificulten innecesariamente la experiencia.

Quality Attributes:

Usability - QA-Priority: 2

Los controles, checkpoints e interacciones con objetos deben ser claros y comprensibles.

Security - QA-Priority: 0

El juego es inicialmente offline y no maneja información personal o sensible del jugador.

Stakeholder Card

Project: **Cat's Odyssey**

Stakeholder: **Owner / Equipo desarrollador**

Goal:

Desarrollar y mantener Cat's Odyssey durante el semestre, procurando que nuevas mecánicas, enemigos y niveles puedan incorporarse sin duplicar innecesariamente el código y manteniendo una ejecución fluida.

Quality Attributes:

Availability - QA-Priority: 1

El juego debe mantenerse utilizable y recuperar correctamente el estado definido por los checkpoints.

Security - QA-Priority: 0

El proyecto no maneja actualmente información sensible ni servicios externos.

Performance - QA-Priority: 2

Movimiento, colisiones, enemigos y objetos deben ejecutarse con fluidez.

Maintainability - QA-Priority: 2

El código debe permitir modificar niveles, enemigos y mecánicas sin cambios innecesarios en todo el sistema.

Concern Card

Concern-ID: **1**

Concern:

Cat's Odyssey tendrá diferentes escenarios. ¿Cómo deben construirse los niveles para poder agregar nuevos escenarios sin duplicar toda la lógica del juego?

Options:

- 1. Crear una clase World diferente y programar individualmente la lógica de cada nivel.
Usability(=) | Security(=) | Availability(=) | Performance(+) | Maintainability(--)
- 2. Crear una estructura base reutilizable para los niveles y configurar en cada escenario plataformas, enemigos, checkpoints y objetos.
Usability(+) | Security(=) | Availability(+) | Performance(+) | Maintainability(++)
- 3. Definir los niveles mediante archivos o configuraciones externas que indiquen qué objetos cargar y dónde ubicarlos.
Usability(=) | Security(=) | Availability(-) | Performance(=) | Maintainability(++)

Concern Card

Concern-ID: **2**

Concern:

Cuando Preciosa muera debe volver al último checkpoint conservando los coleccionables obtenidos. ¿Cómo debemos almacenar y restaurar ese estado?

Options:

1. Guardar únicamente la posición del último checkpoint.
Usability(--) | Security(=) | Availability(-) | Performance(++) | Maintainability(+)
2. Mantener durante la partida un estado del nivel con checkpoint y lista de coleccionables obtenidos.
Usability(++) | Security(=) | Availability(+) | Performance(+) | Maintainability(+)
3. Guardar en un archivo local el checkpoint, los coleccionables y el estado necesario del nivel.
Usability(++) | Security(-) | Availability(++) | Performance(-) | Maintainability(-)

Concern Card

Concern-ID: **3**

Concern:

Cat's Odyssey tendrá enemigos con comportamientos diferentes. ¿Cómo debemos estructurarlos para evitar programar desde cero movimiento, colisiones y lógica común en cada enemigo?

Options:

1. Crear cada enemigo como una clase totalmente independiente.
Usability(=) | Security(=) | Availability(=) | Performance(+) | Maintainability(--)
2. Crear una clase general Enemy y utilizar herencia y polimorfismo para especializar los distintos enemigos.
Usability(+) | Security(=) | Availability(=) | Performance(+) | Maintainability(++)
3. Separar los comportamientos de los enemigos en componentes intercambiables, por ejemplo patrullar, perseguir o volar.
Usability(+) | Security(=) | Availability(=) | Performance(=) | Maintainability(++)

Concern Card

Concern-ID: **4**

Concern:

Cat's Odyssey podrá incorporar hasta dos power-ups. ¿Cómo debemos implementarlos para que agregar o modificar una habilidad no obligue a llenar la clase Player con muchas condiciones específicas?

Options:

1. Programar todos los power-ups directamente en Player utilizando if/else.
Usability(=) | Security(=) | Availability(=) | Performance(++) | Maintainability(--)
2. Mantener los power-ups dentro de Player, pero separarlos en métodos y estados claramente definidos.
Usability(+) | Security(=) | Availability(=) | Performance(+) | Maintainability(+)
3. Crear clases de habilidades independientes que puedan asignarse temporalmente al jugador.
Usability(+) | Security(=) | Availability(=) | Performance(=) | Maintainability(++)

Concern Card

Concern-ID: **5**

Concern:

Cat's Odyssey tendrá introducción, juego, muerte, retorno al checkpoint y final de nivel. ¿Cómo debemos organizar estos estados sin mezclar toda la lógica en una única clase?

Options:

1. Controlar todos los estados mediante variables y múltiples if/else dentro del mismo World.
Usability(-) | Security(=) | Availability(-) | Performance(+) | Maintainability(--)
2. Separar las principales pantallas y estados en diferentes clases World, por ejemplo IntroWorld, LevelWorld y EndingWorld.
Usability(+) | Security(=) | Availability(+) | Performance(+) | Maintainability(+)
3. Crear un administrador de estados que controle transiciones como INTRO, PLAYING, DEAD, CHECKPOINT y VICTORY.
Usability(+) | Security(=) | Availability(+) | Performance(=) | Maintainability(++)

Event Card

Title: **Expansión del videojuego**

Description:

El profesor solicita incorporar tres niveles adicionales sin extender significativamente el tiempo de desarrollo. Las decisiones relacionadas con la construcción y reutilización de niveles deben reconsiderarse. La QA-Priority de Maintainability del Owner aumenta de 2 a 3.

Event Card

Title: **Caída de rendimiento**

Description:

Durante las pruebas, algunos niveles presentan disminuciones de rendimiento cuando hay muchos enemigos y objetos simultáneamente. Deben revisarse las decisiones relacionadas con niveles y enemigos. La QA-Priority de Performance del Owner aumenta de 2 a 3.

Event Card

Title: **Guardado permanente obligatorio**

Description:

Se establece como requisito que el jugador pueda cerrar Cat's Odyssey y continuar posteriormente desde su progreso guardado. El sistema actual de checkpoints debe reconsiderarse. La QA-Priority de Availability del Owner aumenta de 1 a 3.

Event Card

Title: **Nueva habilidad para Preciosa**

Description:

Durante el diseño del nivel final surge la necesidad de agregar una habilidad adicional a las originalmente consideradas. La estructura utilizada para las habilidades debe permitir incorporarla sin modificar excesivamente Player.

Event Card

Title: **Problemas de usabilidad al trepar**

Description:

Durante pruebas con jugadores se detecta que la mecánica de trepar o adherirse a superficies no se comprende fácilmente. Se debe mejorar la claridad de los controles y el feedback visual sin eliminar la mecánica. La QA-Priority de Usability del Usuario aumenta de 2 a 3.